SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:100**
Duration : 1 Hour

Experiments

Code of Conduct:

I _Pusalapati Chandu(192372119)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Pchandu

SIMATS ENGINEERING
ASSESSMENT TOOLS

Sl. No.	Lab Experiments - Questions
1	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → 0A1 A → 0A | 1A | ε
2	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → 0S0 | 1S1 | 0 | 1 | ε
3	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → 0S0 | A A → 1A | ε
4	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → 0S1 | ε
5	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → A101A A → 0A | 1A | ε

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOLS

Experiment 1 — S → 0A1, A → 0A | 1A | ε

Step 1: Work out what language this grammar generates

S always produces a leading 0, then whatever A produces, then a trailing 1. A can keep adding a 0 or a 1 in front of itself any number of times, or stop immediately (ε). So A can generate any string of 0s and 1s at all, including the empty string. Putting it together: L(G) = { 0 w 1 : w is any string over {0,1} }, which in regular-expression form is 0(0|1)*1. In words: the string must start with 0, end with 1, and have length at least 2 (the shortest string is '01', when A produces ε).

Step 2: Design the checking logic

Since this is a regular language, we don't need a stack or counters — we just need to confirm the first symbol is 0, the last symbol is 1, the string is at least 2 symbols long, and every symbol is 0 or 1. To keep the program honestly 'grammar-driven' (useful for showing your working in the lab), the code below still writes a parseA() function that recurses exactly the way the grammar's A does — consuming symbols one at a time until only the final required '1' is left — rather than just doing a one-line string check.

Step 3: C program

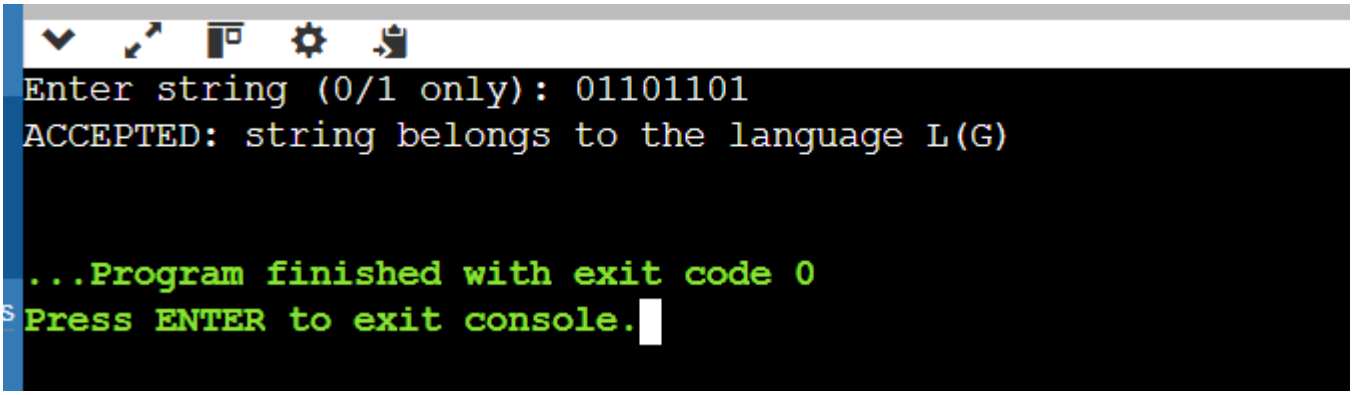
```
#include <stdio.h>
#include <string.h>

char str[1000];
int len;

/* A -> 0A | 1A | epsilon                                          */
/* A must leave exactly ONE character unconsumed, because S needs */
/* a final '1' right after A finishes.                             */
int parseA(int pos) {
    if (pos == len - 1) return 1;   /* epsilon: one char left for the final 1 */
    if (pos >= len) return 0;       /* ran out of string too early            */
    if (str[pos] == '0' || str[pos] == '1')
        return parseA(pos + 1);     /* consume one symbol, recurse            */
    return 0;
}

/* S -> 0 A 1 */
int parseS() {
    if (len < 2) return 0;
    if (str[0] != '0') return 0;
    if (str[len - 1] != '1') return 0;
    return parseA(1);
}

int main() {
    printf("Enter string (0/1 only): ");
    scanf("%s", str);
```

```
    len = strlen(str);
    if (parseS())
        printf("ACCEPTED: string belongs to the language L(G)\n");
    else
        printf("REJECTED: string does NOT belong to the language L(G)\n");
    return 0;
}
```

Step 4: Dry run (compiled and tested)

Input string	Result	Why
01	ACCEPTED	starts 0, ends 1, A produced ε
0100101	ACCEPTED	starts 0, ends 1, middle is any mix
0110	REJECTED	ends in 0, not 1
10	REJECTED	does not start with 0
00	REJECTED	ends in 0, not 1

Output:



This is a regular language (equivalent to the regular expression 0(0|1)*1), so it could just as easily be recognised by a finite automaton — the recursive C function above is simply the CFG's own structure translated directly into code.

Experiment 2 — S → 0S0 | 1S1 | 0 | 1 | ε

Step 1: Work out what language this grammar generates

Every time S expands via 0S0 or 1S1, it glues an identical symbol onto both the front and the back of whatever S produces next, then S eventually bottoms out with ε (even-length case) or a single 0 or 1 (odd-length case). Wrapping a string symmetrically like this, over and over, is exactly the definition of a palindrome. So L(G) = the set of all binary palindromes, including the empty string — for example ε, 0, 1, 11, 1001, 10011001.

Step 2: Design the checking logic

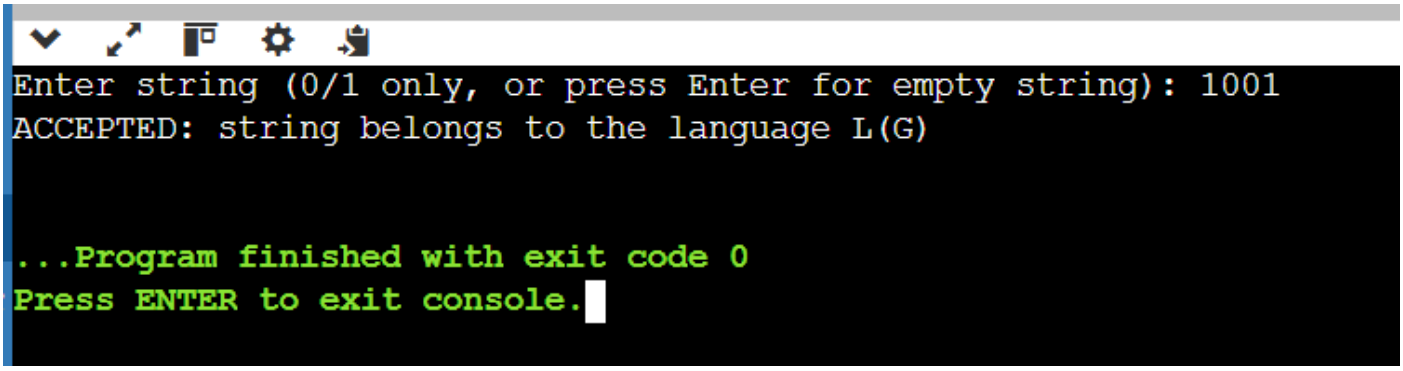
The grammar's own recursive shape — match the outer pair, then recurse on what's inside — is the cleanest possible algorithm here: keep two pointers, one from the left and one from the right, and move them inward one step at a time as long as the characters they point to match. If the pointers cross or meet, the string is a palindrome; if the characters ever disagree, it isn't. This is a direct translation of 0S0 / 1S1 (match outer pair, recurse inward) and the base cases S → ε / S → 0 / S → 1 (pointers have crossed, or point to the same single middle character).

Step 3: C program

```
#include <stdio.h>
#include <string.h>

char str[1000];

int isPalindromeCFG(int left, int right) {
    if (left > right)  return 1;   /* S -> epsilon : nothing left to check   */
    if (left == right) return 1;   /* S -> 0 | 1   : single middle symbol    */
    if (str[left] != str[right]) return 0;   /* outer pair must match        */
    return isPalindromeCFG(left + 1, right - 1); /* S->0S0 or 1S1, shrink in */
}

int main() {
    printf("Enter string (0/1 only, or press Enter for empty string): ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';   /* strip the newline fgets keeps */
    int n = strlen(str);
    if (isPalindromeCFG(0, n - 1))
        printf("ACCEPTED: string belongs to the language L(G)\n");
    else
        printf("REJECTED: string does NOT belong to the language L(G)\n");
    return 0;
}
```

Step 4: Dry run (compiled and tested)

Input string	Result	Why
(empty)	ACCEPTED	S → ε directly
1001	ACCEPTED	reads the same forwards and backwards
10011001	ACCEPTED	palindrome of length 8
1000	REJECTED	reverse is 0001, does not match



Worth noting for the 'conceptual understanding' part of the rubric: unlike Experiment 1, the language of all binary palindromes is NOT regular — it can be proved non-regular with the pumping lemma (informally, a finite automaton has no way to 'remember' an arbitrarily long first half of the string in order to check it against the second half). It is, however, context-free, which is exactly why this grammar can generate it while no regular expression can.

Experiment 3 — S → 0S0 | A, A → 1A | ε

Step 1: Work out what language this grammar generates

S wraps a 0 on both ends, n times, exactly like the previous grammar, but instead of bottoming out at ε or a single symbol, it hands off to A, and A can only generate a run of 1s (or nothing). So the string always looks like: n zeros, then some number of ones, then the same n zeros again. Formally, L(G) = { $0^n 1^m 0^n$: $n \geq 0, m \geq 0$ } — for example 0110, 001100, or just 111 (n = 0).

Step 2: Design the checking logic

Peel matching 0s off the front and back together (this is 0S0 applied repeatedly) and count how far in you get; whatever is left in the middle must satisfy A, i.e. be made up of 1s only (or be empty). Two pointers moving inward from both ends do this in one pass.

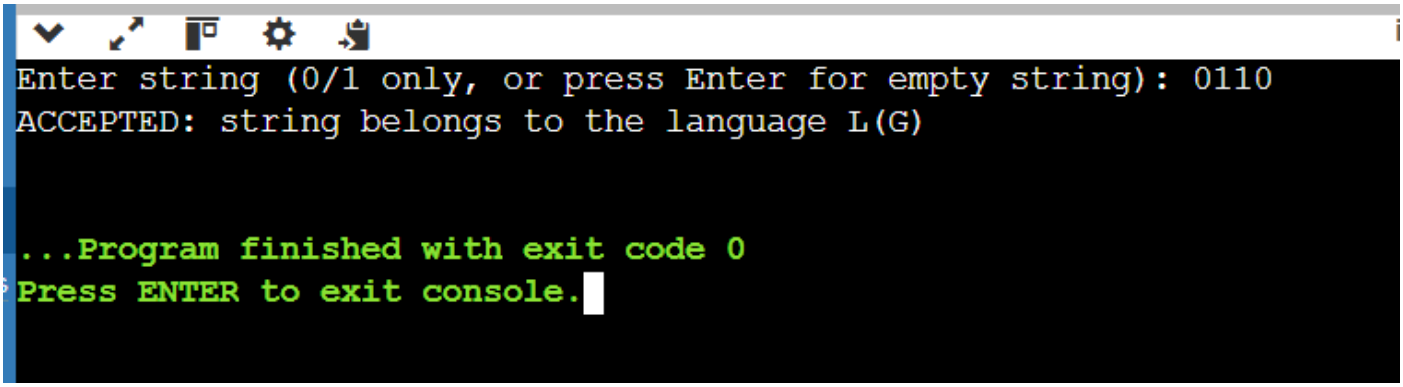
Step 3: C program

```
#include <stdio.h>
#include <string.h>

char str[1000];

int main() {
    printf("Enter string (0/1 only, or press Enter for empty string): ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    int n = strlen(str);

    int left = 0, right = n - 1;
    /* S -> 0S0 applied repeatedly: strip a matching 0 from each end */
    while (left < right && str[left] == '0' && str[right] == '0') {
```

```
        left++;
        right--;
    }

    /* whatever remains (A -> 1A | epsilon) must be all 1s, or nothing */
    int accept = 1;
    for (int i = left; i <= right; i++) {
        if (str[i] != '1') { accept = 0; break; }
    }

    if (accept)
        printf("ACCEPTED: string belongs to the language L(G)\n");
    else
        printf("REJECTED: string does NOT belong to the language L(G)\n");
    return 0;
}
```

Step 4: Dry run (compiled and tested)

Input string	Result	Why
00	ACCEPTED	n=1, m=0 → $0^1 1^0 0^1$
0110	ACCEPTED	n=1, m=2 → $0^1 1^2 0^1$
001100	ACCEPTED	n=2, m=2 → $0^2 1^2 0^2$
111	ACCEPTED	n=0, m=3 → $0^0 1^3 0^0$
0	REJECTED	a single 0 can't split into equal front/back counts
0010	REJECTED	front run of 0s (2) ≠ back run of 0s (1)



This language is context-free but not regular — proving it with the pumping lemma is a standard exercise, since a finite automaton would need to count the leading zeros and still remember that count arbitrarily far later to check the trailing zeros, which requires unbounded memory.

Experiment 4 — S → 0S1 | ε

Step 1: Work out what language this grammar generates

S wraps a 0 on the left and a 1 on the right, n times, then stops with ε. So the string is always n zeros followed immediately by n ones, with nothing else and nothing out of order. L(G) = { $0^n 1^n : n \geq 0$ } — this is probably the single most famous example in the whole course, since it's the textbook string used to demonstrate the pumping lemma for regular languages.

Step 2: Design the checking logic

Count the leading run of 0s, then count the run of 1s that follows, then check two things: nothing is left over afterwards (no stray characters, and no 1s appearing before a 0 or vice versa), and the two counts are equal.

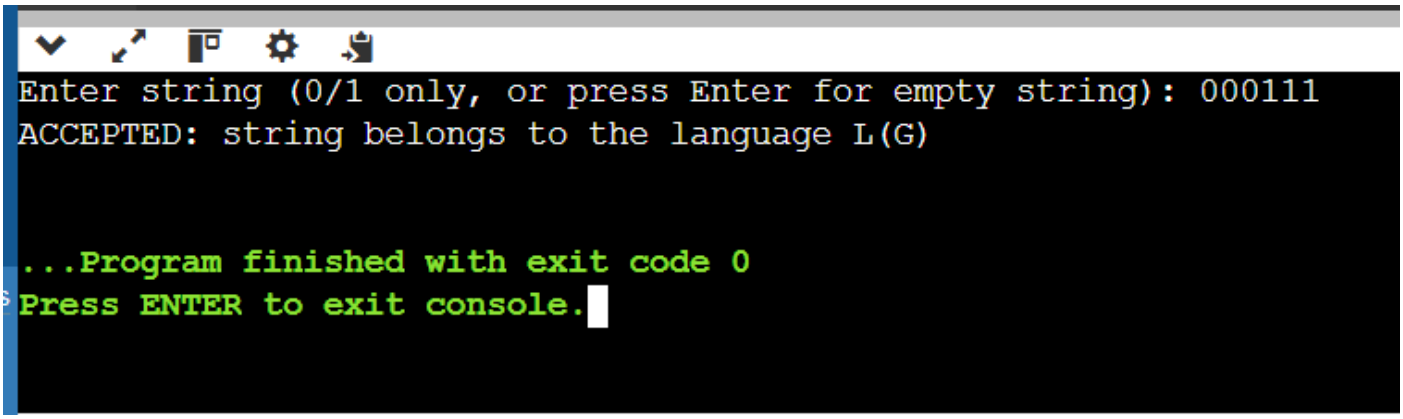
Step 3: C program

```
#include <stdio.h>
#include <string.h>

char str[1000];

int main() {
    printf("Enter string (0/1 only, or press Enter for empty string): ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    int n = strlen(str);

    int i = 0, zeros = 0;
    while (i < n && str[i] == '0') { zeros++; i++; }  /* S -> 0S applied 'zeros' times */

    int ones = 0;
    while (i < n && str[i] == '1') { ones++; i++; }    /* the matching 1s */

    int accept = (i == n) && (zeros == ones); /* nothing left over, counts match */

    if (accept)
        printf("ACCEPTED: string belongs to the language L(G)\n");
    else
        printf("REJECTED: string does NOT belong to the language L(G)\n");
    return 0;
}
```

Step 4: Dry run (compiled and tested)

Input string	Result	Why
(empty)	ACCEPTED	n=0

01	ACCEPTED	n=1
000111	ACCEPTED	n=3
0001	REJECTED	3 zeros but only 1 one
011	REJECTED	1 zero but 2 ones
0110	REJECTED	characters out of order (a 0 appears after a 1)



This is the canonical non-regular, context-free language: a finite automaton has only a fixed number of states to work with, so it cannot count an unbounded number of leading 0s and still verify that exact count of 1s follows — the pumping lemma formalises exactly why that's impossible. A pushdown automaton (which this CFG corresponds to) solves it easily with a stack: push on every 0, pop on every 1, accept if the stack is empty exactly when the input ends.

Experiment 5 — S → A101A, A → 0A | 1A | ε

Step 1: Work out what language this grammar generates

A can generate any string over {0,1} at all (including the empty string) — it has no restriction, unlike the A in Experiment 3. S sandwiches the fixed substring '101' between two independent copies of A. So L(G) = { w : w contains '101' as a substring somewhere }, i.e. every binary string that has the pattern 101 occurring anywhere inside it. In regular-expression form: (0|1)* 101 (0|1)*.

Step 2: Design the checking logic

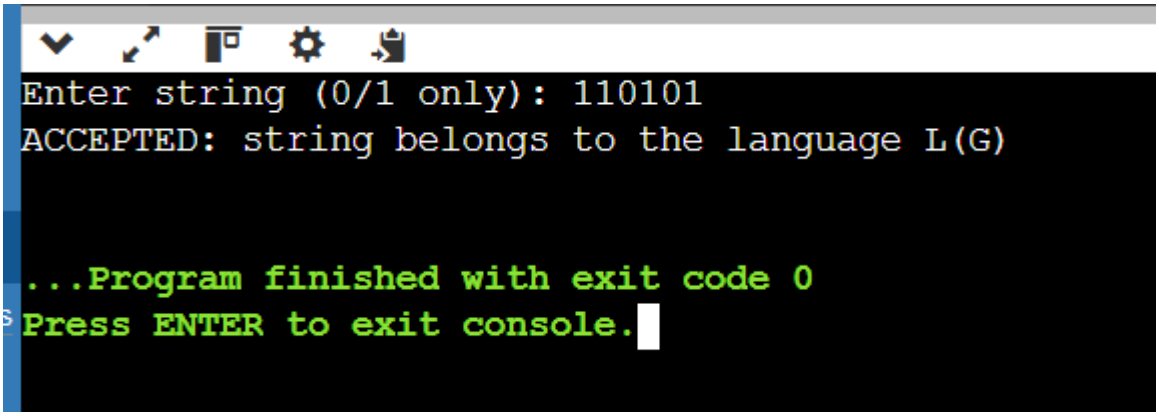
Since A places no constraint on what comes before or after, the only real requirement is a substring search for the pattern '101' anywhere in the input — a straightforward sliding window that checks three consecutive characters at a time.

Step 3: C program

```
#include <stdio.h>
#include <string.h>

char str[1000];

/* returns 1 if "101" occurs anywhere inside s */
int containsPattern(char *s) {
    int n = strlen(s);
    for (int i = 0; i + 3 <= n; i++) {
        if (s[i] == '1' && s[i+1] == '0' && s[i+2] == '1')
            return 1;
    }
    return 0;
}

int main() {
    printf("Enter string (0/1 only): ");
    scanf("%s", str);
    if (containsPattern(str))
        printf("ACCEPTED: string belongs to the language L(G)\n");
    else
        printf("REJECTED: string does NOT belong to the language L(G)\n");
    return 0;
}
```

Step 4: Dry run (compiled and tested)

Input string	Result	Why
101	ACCEPTED	the whole string is the pattern
110101	ACCEPTED	contains 101 starting at index 2
1010	ACCEPTED	contains 101 starting at index 0
1001	REJECTED	no run of 1-0-1 anywhere
000	REJECTED	no 1s at all
111000	REJECTED	no 1-0-1 pattern present



This language is regular (it's exactly what the regular expression (0|1)*101(0|1)* describes), and in fact it's also exactly the kind of language a 4-state DFA is built for in class — one state per 'how much of 101 have I matched so far'.

Quick summary across all five experiments

Grammar	Language	Regular?
S → 0A1, A → 0A|1A|ε	0(0|1)*1	Yes
S → 0S0|1S1|0|1|ε	binary palindromes	No
S → 0S0|A, A → 1A|ε	$0^n1^m0^n$	No
S → 0S1|ε	0^n1^n	No
S → A101A, A → 0A|1A|ε	contains substring 101	Yes

The two regular ones (1 and 5) could be checked with a simple pattern/scan because a finite amount of 'memory' (a fixed lookahead or a few states) is enough. The three non-regular ones (2, 3, 4) all needed some form of counting or matching-from-both-ends, because verifying them requires remembering an unbounded amount of information about what's already been seen — which is precisely the intuition the pumping lemma turns into a formal proof technique.